

Private Constructors in C# with Examples

In this article, I am going to discuss **Private Constructors in C#** with Examples. I got many emails, messages, and comments to make an article related to Private Constructors in C#. There are a lot of confusion and doubts regarding private constructors among the developers and I hope at the end of this article, all your doubts and confusions will be cleared.

What is a Private Constructor in C#?

In C#, when the constructor is created by using the **Private Access Specifier**, then it is called a **Private Constructor**. When a class contains a private constructor and if the class **does not have any other Public Constructors**, then you **cannot create an object for the class outside of the class**. But we can **create objects for the class within the same class**.

So, the most important point that we need to keep in mind is that the restrictions such as restrictions for creating objects, restrictions for accessing members, etc. will come into the picture when you are trying to create the objects and accessing the members from outside the class. If you are creating the objects and accessing the members within the same class, then no such restrictions come into the picture.

Creating Object using Private Constructor within the same class in C#:

Many articles on the web say that you cannot create an instance of the class if it has a private constructor. But this is partially true. You cannot create an instance from outside the class, but you can create the instance from within the class. For a better understanding, please have a look at the below example. Here, in the Program class, we defined a private constructor and the Main method is also defined in the same Program class. As you can see, within the Main method, we are creating an instance of the Program class and calling the Method1.

```
using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        //Private Constructor
        private Program()
        {
            Console.WriteLine("This is Private Constructor");
        }
        public void Method1()
        {
            Console.WriteLine("Method1 is Called");
        }
        static void Main(string[] args)
        {
            //Creating instance of Program class using Private Constructor
            Program obj = new Program();
            obj.Method1();
            Console.ReadKey();
        }
    }
}
```

```
}  
}
```

Output:

```
This is Private Constructor  
Method1 is Called
```

Creating an Instance from Outside the Class in C#:

The point that you need to remember is while creating the instance from outside the class, the class should have a public constructor. It does not matter if a class has a private constructor or not, but if a class has a public constructor, then using that public constructor, we can create the class instance and invoke the public non-static members.

For a better understanding, please have a look at the below example. Here, we have the Test class with both Private and Public constructors and a public method. Now, from the Program class Main method (which is outside of the Test class), we are creating an instance of the Test class and invoking the Method1.

```
using System;  
namespace PrivateConstructorDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating instance of Test class using public Constructor  
            Test obj = new Test(10);  
            obj.Method1();  
            Console.ReadKey();  
        }  
    }  
    public class Test  
    {  
        //Private Constructor  
        private Test()  
        {  
            Console.WriteLine("This is Private Constructor");  
        }  
        //Public Constructor  
        public Test(int x)  
        {  
            Console.WriteLine("This is public Constructor");  
        }  
        public void Method1()  
        {  
            Console.WriteLine("Method1 is Called");  
        }  
    }  
}
```

```
}
```

Output:

```
This is public Constructor  
Method1 is Called
```

Now, what happens if the class does not have any public constructor and has only a private constructor? Then we cannot create an instance from outside the class. For a better understanding, please have a look at the below example. Here, the Test class is having only a private constructor and from the Main method, while creating the instance of the Test class using the Private constructor, we are getting a compile-time error.

```
using System;  
namespace PrivateConstructorDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating instance of Test class using public Constructor  
            Test obj = new Test();  
            obj.Method1();  
            Console.ReadKey();  
        }  
    }  
    public class Test  
    {  
        //Private Constructor  
        private Test()  
        {  
            Console.WriteLine("This is Private Constructor");  
        }  
  
        public void Method1()  
        {  
            Console.WriteLine("Method1 is Called");  
        }  
    }  
}
```

Output:

	Code	Description
	CS0122	'Test.Test()' is inaccessible due to its protection level

As you can see here, we are getting the error **'Test.Test()' is inaccessible due to its protection level** and this makes sense because the Test class Test() constructor is inaccessible in the Program class because of

its protection level i.e. private.

Note: The first important point that you need to remember is Private constructor restricts the class to be instantiated from outside the class only if it does not have any public constructor. If it has a public constructor, then we can create the instance from outside of the class. There is no restriction to creating the instance from within the same class.

Use Case: The use case of Private Constructor in C# is that if you don't want your class to be instantiated from outside the class, then add a private constructor without any public constructor in your class.

Private Constructor Restricting Inheritance in C#:

On many websites, you will find that they are saying Private Constructor Restricting Inheritance in C#. That means if you have a private constructor in a class, then that class cannot be inherited. This is also partially true. Let us prove this point with a few examples.

See, if your class has a private constructor as well as any other public constructor, then it is possible to inherit your class. What the child class requires to establish a parent-child relationship is a publicly accessible constructor. That means it does not matter whether your class has any private constructor or not, if it has a public constructor, then you can inherit the class.

For a better understanding, please have a look at the following example. Here, inside the Parent class, we have two constructors i.e. one private constructor and one public constructor. Then Child class inherits from the Parent class and we are not getting any errors.

```
using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating instance of Child class
            Child obj = new Child();
            Console.ReadKey();
        }
    }
    public class Parent
    {
        //Private Constructor
        private Parent()
        {
            Console.WriteLine("Parent Class Private Constructor is Called");
        }
        //Public Constructor
        public Parent(string Message)
        {
            Console.WriteLine("Parent Class Public Constructor is Called");
        }
    }
}
```

```

public class Child : Parent
{
    public Child() : base ("Hello")
    {
        Console.WriteLine("Child Class Public Constructor is Called");
    }
}

```

Output:

```

Parent Class Public Constructor is Called
Child Class Public Constructor is Called

```

If the Parent class has only a private constructor and does not have any public constructors then Inheritance is not possible with outside classes. For a better understanding, please have a look at the following example. Here, the Parent class has only a private constructor hence inheritance is not possible and this time we will get a compile-time error.

```

using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating instance of Child class
            Child obj = new Child();
            Console.ReadKey();
        }
    }
    public class Parent
    {
        //Private Constructor
        private Parent()
        {
            Console.WriteLine("Parent Class Private Constructor is Called");
        }
    }
    public class Child : Parent
    {
        public Child()
        {
            Console.WriteLine("Child Class Public Constructor is Called");
        }
    }
}

```

Output:

	Code	Description
	CS0122	'Parent.Parent()' is inaccessible due to its protection level

Here we are getting the compile-time error and this makes sense because there is no publicly accessible constructor in the Parent class which the Child class constructor can access.

In the above example, the Child class is from outside the Parent class and hence it is not accessible to the Parent class private constructor. But if the child class is an Inner class of the Parent class, then inheritance is possible. For a better understanding, please have a look at the below example. Here, the Child class is declared inside the Parent class and inheritance is possible.

```
using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating instance of Child class
            Parent.Child obj = new Parent.Child();
            Console.ReadKey();
        }
    }
    public class Parent
    {
        //Private Constructor
        private Parent()
        {
            Console.WriteLine("Parent Class Private Constructor is Called");
        }
        public class Child : Parent
        {
            public Child()
            {
                Console.WriteLine("Child Class Public Constructor is Called");
            }
        }
    }
}
```

Output:

```
Parent Class Private Constructor is Called
Child Class Public Constructor is Called
```

If you want to restrict inheritance then don't go for the private constructor, instead, make the **Class Sealed** which will restrict the class to be inherited from either outside or inside. For a better understanding, please have a look at the following example.

```

using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating instance of Child class
            Parent.Child1 obj1 = new Parent.Child1();
            Child2 obj2 = new Child2();
            Console.ReadKey();
        }
    }
    public sealed class Parent
    {
        //Private Constructor
        private Parent()
        {
            Console.WriteLine("Parent Class Private Constructor is Called");
        }
        public class Child1 : Parent
        {
            public Child1()
            {
                Console.WriteLine("Child1 Class Public Constructor is Called");
            }
        }
    }
    public class Child2 : Parent
    {
        public Child2()
        {
            Console.WriteLine("Child2 Class Public Constructor is Called");
        }
    }
}

```

Output:

	Code	Description
	CS0509	'Parent.Child1': cannot derive from sealed type 'Parent'
	CS0509	'Child2': cannot derive from sealed type 'Parent'

So, we prove that the private constructor does not restrict inheritance in C#.

Use Private Constructor when your class has only Static Members in C#:

On the Internet, you will find that many peoples or articles are saying that, if your class contains only static members, then make the class constructor private. But this is wrong. I have already shown you that even if you have a private constructor then we can also make inheritance with inner classes and we can also create instances within the same class.

See, the constructors either private or public are used to create the instances and initialize the non-static variables of a class. If your class does not have any non-static members, then is there any requirement for a constructor? The answer is No. If your class contains only static members, then it is better to make the class static which not only restricts your class instance to be created but also it will restrict inheritance.

For a better understanding, please have a look at the below example. Here, we have a static class called Test with two static data members. As you can see we are getting compiler time errors the Child class trying to inherit from the Test class as well as we are also getting compile-time errors while creating the instance of the static Test class.

```
using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Cannot Create an instance of the Static class
            Test test = new Test();
            Console.ReadKey();
        }
    }
    public static class Test
    {
        public static double PI = 3.14;
        public static int GetSquare(int x)
        {
            return x * x;
        }
    }
    //A class cannot Derive from a Static Class
    public class Child: Test
    {
    }
}
```

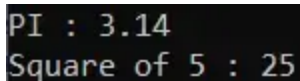
Output:

	Code	Description
	CS0709	'Child': cannot derive from static class 'Test'
	CS0723	Cannot declare a variable of static type 'Test'
	CS0712	Cannot create an instance of the static class 'Test'

Then you might have one question on your mind, what is the use of static class, and how we can access the static members as we are unable to create the instance as well as unable to make the inheritance relationship? You can access the static members of a class directly by using the class name and static members get memory allocation only once i.e. only one copy of the static members is available and that is too for the first time when the execution start. For a better understanding, please have a look at the below example. Here, we are accessing the two static members using the class name Test.

```
using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"PI : {Test.PI}");
            Console.WriteLine($"Square of 5 : {Test.GetSquare(5)}");
            Console.ReadKey();
        }
    }
    public static class Test
    {
        public static double PI = 3.14;
        public static int GetSquare(int x)
        {
            return x * x;
        }
    }
}
```

Output:

A screenshot of a console window showing the output of the program. The first line is "PI : 3.14" and the second line is "Square of 5 : 25".

```
PI : 3.14
Square of 5 : 25
```

Is Private Constructor going to be Parameterless in C#?

Again, in many articles and YouTube videos, you will see that many peoples are saying private constructors should be parameterless. But this is not true. See, in C#, constructors are overloaded. That means we can define multiple constructors with different parameters. In overloading what is important is the name and the parameters and it does not consider access specifiers like private, public, protected, etc as part of the overloading. So, it is possible to define multiple private constructors in C#.

For a better understanding, please have a look at the below example. Here, in the Program class, we define two private constructors. One private constructor is without parameters and another private constructor is with parameters. As you can see, here we are creating two instances inside the Main method using both the private constructors.

```
using System;
namespace PrivateConstructorDemo
{
```

```

class Program
{
    private Program()
    {
        Console.WriteLine("Private Parameterless Constructor is Called");
    }
    private Program(string message)
    {
        Console.WriteLine("Private Parameterized Constructor is Called");
    }
    static void Main(string[] args)
    {
        Program obj1 = new Program();
        Program obj2 = new Program("Hello");
        Console.ReadKey();
    }
}

```

Output:

```

Private Parameterless Constructor is Called
Private Parameterized Constructor is Called

```

When to use Private Constructors in C#?

On Internet, you will find many articles and many peoples are saying that a private constructor is used to implement Singleton Design Pattern. Yes, absolutely right. One of the use cases of the private constructors is to implement a singleton design pattern. Let us see how to implement a singleton design pattern using a private constructor in C#.

What is Singleton Pattern in C#?

The Singleton Design Pattern ensures that only one instance of a particular class is going to be created and then provide simple global access to that instance for the entire application.

How to Implement Singleton Design Pattern in C#?

The following are the steps to implement the Singleton Design Pattern in C#.

1. You need to declare only a single constructor in your class and that constructor should be private and parameterless. This is required because it is not allowed the class to be instantiated from outside the class. It only instantiates from within the class.
2. The class should be declared sealed which will ensure that it cannot be inherited.
3. You need to create a private static variable that is going to hold a reference to the single created instance of the class.
4. You also need to create a public static property/method which will return the single-created instance of the singleton class. This method or property first checks if an instance of the singleton class is

available or not. If the singleton instance is available, then it returns that singleton instance otherwise it will create an instance and then return that instance.

Singleton Design Pattern Implementation Example in C#:

```
using System;
namespace PrivateConstructorDemo
{
    public sealed class Singleton
    {
        private static int counter = 0;
        private static Singleton instance = null;
        private static readonly object Instancelock = new object();
        public static Singleton GetSingletonInstance()
        {
            lock (Instancelock)
            {
                if (instance == null)
                {
                    instance = new Singleton();
                }
                return instance;
            }
        }
        private Singleton()
        {
            counter++;
            Console.WriteLine($"Singleton Constructor Executed {counter} Time'
        }
        public void SomeMethod(string Message)
        {
            Console.WriteLine($"Some Method Called : {Message}");
        }
    }
}
```

Next, modify the Main method of the Program class as follows:

```
using System;
namespace PrivateConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Singleton fromPlace1 = Singleton.GetSingletonInstance();
            fromPlace1.SomeMethod("From Place 1");
            Singleton fromPlace2 = Singleton.GetSingletonInstance();
            fromPlace2.SomeMethod("From Place 2");
            Console.ReadKey();
        }
    }
}
```

```
}  
}  
}
```

Output:

```
Singleton Constructor Executed 1 Time  
Some Method Called : From Place 1  
Some Method Called : From Place 2
```

If you want to learn Singleton Design Pattern in detail, please click on the below link.

<https://dotnettutorials.net/lesson/singleton-design-pattern/>

In the next article, I am going to discuss [Destructor in C#](#) with Examples. Here, in this article, I try to explain **Private Constructors in C#** with Examples. I hope you enjoy this Private Constructor in C# with Examples article. Please give your feedback, suggestions, and questions about Private Constructors in the C# article in the comment section.

Related Articles:

[Constructors in C#](#)

[Types of Constructors in C#](#)

[When to use Constructors in C# Application Development](#)

[Static vs Non-Static Constructors in C#](#)